Torneo de Komikan Concurrente — Informe de diseño

Sistemas Operativos · Tarea 2 · Cristian Hernández y Diego Soto

1. Diseño del esquema de sincronización

Hilos. El proceso tiene un hilo de señales, N hilos cliente (llegada concurrente), el hilo principal que actúa como Matchmaker y, por cada mesa activa, un hilo Juez que lanza 13 hilos de ficha (1 Puma y 12 Perros). Todos se unen con `pthread_join`.

Tablero y exclusión mutua. Cada mesa tiene un único `pthread_mutex_t` que protege el tablero (ocupación de los 28 nodos y posición de cada ficha) y el estado de la partida. Una ficha duerme su retardo fuera de la sección crítica, y al despertar ejecuta de forma atómica «validar y aplicar» el movimiento: comprueba adyacencia, casilla libre o salto válido, modifica el tablero, incrementa el contador y evalúa la victoria. Si dos fichas apuntan a la misma casilla, la que entra primero al mutex la ocupa; la segunda encuentra la casilla ocupada, su intento falla sin efectos y reintenta tras un nuevo retardo. Así no existen estados intermedios visibles ni dos fichas en una casilla.

Variables de condición. Cada mesa usa un `pthread_cond_t` con reloj `CLOCK_MONOTONIC`. Las fichas duermen con `pthread_cond_timedwait`, que libera el mutex durante la espera, evita la espera activa y permite despertarlas todas de inmediato con un `broadcast` al terminar, cancelar o caer una ficha. El Juez espera con `pthread_cond_wait` hasta que haya victoria, abandono o cancelación.

Límite de mesas. Un `sem_t` inicializado en M cuenta las mesas libres. El Matchmaker hace `sem_wait` antes de abrir cada mesa, así que con las M ocupadas espera de forma pasiva; al terminar, cada mesa hace `sem_post`. El Matchmaker revisa la bandera de detención y registra la mesa en una misma sección crítica del servidor, de modo que un SIGINT nunca deja una mesa sin notificar.

Resiliencia. Si una ficha cae (simulado con `DISCONNECT_*`), marca su bandera, hace `broadcast` y sale del bucle sin retener ningún lock, porque solo los toma dentro del bucle y siempre los libera antes de salir. El Juez detecta la anomalía, declara ganador al oponente por abandono, une todos los hilos y recién entonces destruye el mutex y la condición de la mesa. Para Ctrl+C, SIGINT y SIGUSR1 se bloquean antes de crear cualquier hilo y las atiende un hilo dedicado con `sigwait` (nada se hace dentro de un handler asíncrono). Ese hilo marca la detención, llama a `mesa_cancelar` en cada mesa activa y despierta al Matchmaker; luego el programa une todos los hilos, destruye semáforo, mutex y condiciones, libera la memoria y termina con código 130.

2. Análisis de riesgos de concurrencia

Riesgo	Escenario	Estrategia
Deadlock	Hilos que toman varios locks en órdenes distintos (servidor, mesa, log).	Orden global fijo: mutex del servidor → mutex de mesa → mutex del log. Las fichas solo toman el de su mesa y el del log; la mesa nunca toma el del servidor. Dentro de la mesa hay un solo lock, así que no hay espera circular entre casillas.
Livelock	Dos fichas que se disputan siempre la misma casilla y reintentan al unísono.	No hay protocolo de cesión mutua: el perdedor falla y espera un retardo aleatorio nuevo (`rand_r` con semilla propia por hilo). Además el Puma prioriza capturar y los perros eligen entre varios destinos, por lo que la contención no se repite en sincronía.
Starvation	Una ficha que nunca consigue el mutex, o parejas que nunca reciben mesa.	La sección crítica dura microsegundos frente a retardos de 50–300 ms, así que la probabilidad de contención sostenida es mínima. El Matchmaker usa colas FIFO por rol y el semáforo despierta a un solo esperador. `MAX_MOVS` garantiza que toda partida termina y libera su mesa.

Lost wake-up / cierre	Un hilo duerme justo cuando se cancela la partida.	Todas las banderas (`terminada`, `cancelar`, caídas) se leen y escriben con el mutex tomado y se espera siempre dentro de un `while`, por lo que ninguna señal se pierde.

3. Evaluación de las decisiones de diseño

- **Mutex único por mesa (grano grueso) en vez de uno por casilla.** Es lo más simple de razonar y demostrar correcto: la validación de un salto lee tres nodos y la captura modifica dos, y con locks por casilla habría que adquirirlos en orden y evitar deadlock. El costo en concurrencia es bajo: dentro de una mesa solo hay 13 fichas con retardos de decenas de ms y secciones críticas de microsegundos, y las mesas son totalmente independientes entre sí.
- **Rendimiento.** Cada movimiento cuesta O(1); la detección de Puma acorralado es O(grado²), con grado máximo 6. En una prueba de estrés con retardo de 1 ms (N=20, M=4, hasta 500 movimientos por partida) las 10 partidas terminaron en 0,12 s incluso bajo ThreadSanitizer, sin advertencias. Los retardos se cumplen con `timedwait`, sin consumir CPU mientras se duerme.
- **Hilo de señales con `sigwait`.** Evita las restricciones de async-signal-safety de un handler (no se pueden usar mutex ni `free` dentro de él) a cambio de un hilo adicional, y permite una cancelación ordenada.
- **Verificación.** `-fsanitize=thread` sin advertencias, `valgrind --leak-check=full` sin fugas (también tras SIGINT), límite de M mesas comprobado con N=20, M=2, y abandonos de Puma y Perros que dan la victoria al oponente.
- **Limitación asumida.** El tablero tiene una única sección crítica por mesa; si se quisiera escalar a cientos de fichas por mesa, convendría pasar a locks por casilla ordenados o a una cola de movimientos atendida por el Juez.